

CPU Scheduling Comparison Project: Priority vs SRTF

1. Introduction

This project presents a comparative study between two CPU scheduling algorithms: Priority Scheduling and Shortest Remaining Time First (SRTF). The main objective is to analyze how different scheduling policies affect system performance, fairness, and process handling.

Priority Scheduling selects processes based on assigned priority values, making it suitable for policy-driven systems where certain tasks are more important than others. On the other hand, SRTF selects the process with the shortest remaining execution time, aiming to optimize efficiency and minimize waiting time.

This report evaluates both algorithms using multiple test scenarios, including cases of conflicting priorities, starvation-sensitive workloads, and validation cases. The comparison focuses on performance metrics such as Waiting Time (WT), Turnaround Time (TAT), and Response Time (RT), aiming to highlight the trade-offs between efficiency and fairness.

2. Algorithms Description & Assumptions

General Assumptions

- All processes are assumed to have valid (positive, integer) inputs for arrival time, burst time, and priority.
- Lower priority value indicates higher priority in Priority Scheduling.
- Both Priority and SRTF are implemented as preemptive algorithms, where preemption occurs immediately without delay.
- Tie-breaking rule: If two processes have the same priority or remaining burst time, FCFS (First Come First Served) based on arrival time is used.
- Context switching is assumed to take no time (zero overhead).
- Unit-Time step = 1. Time is assumed to be discrete and measured in unit time steps for simulation purposes.
- All processes are available at runtime before the simulation starts.
- Processes are independent and do not involve I/O operations or blocking states.
- CPU is always available and there are no external interruptions or hardware failures.
- Performance metrics (WT, TAT, RT) are calculated using standard operating system formulas:
 - **Turnaround Time (TAT)** = Completion Time - Arrival Time
 - **Waiting Time (WT)** = Turnaround Time - Burst Time
 - **Response Time (RT)** = First CPU Allocation Time - Arrival Time

Limitations

- This project is only a simulation of CPU scheduling and does not represent a real operating system.
- Context switching is assumed to take no time (zero overhead).
- All processes are assumed to be known before the simulation starts.

Algorithm A: Priority Scheduling

Priority Scheduling selects the process with the highest priority to execute.

1. **Rule:** Lower priority value number indicates a higher priority (e.g., 1 is higher priority than 2).
2. **Preemption:** This implementation is preemptive. A newly arriving higher-priority process will immediately interrupt the currently running process.
3. **Tie-Breaking:** If two processes have the same priority, First Come First Served (FCFS) is used based on Arrival Time.
4. **Aging Mechanism (Starvation Prevention):** To prevent processes from waiting forever, an **Aging** mechanism was implemented. For every 5 time units a process spends waiting in the ready queue, its priority value is decreased by 1 (thereby increasing its actual priority towards a maximum of 1).
 - **Advantage:** Ensures important, urgent processes are executed first, while the aging mechanism ensures background tasks eventually get completed.
 - **Disadvantage:** Even with aging, both algorithms may suffer from starvation under certain workloads if maximum-priority tasks continuously arrive.

Algorithm B: Shortest Remaining Time First (SRTF)

SRTF selects the process with the smallest remaining burst time.

1. **Rule:** The CPU is allocated to the process that needs the least amount of time to finish.
2. **Preemption:** This implementation is preemptive. Immediate switching occurs when a new job arrives with a shorter burst time than what is remaining on the current job.
3. **Tie-Breaking:** If two processes have the exact same remaining time, First Come First Served (FCFS) is used based on Arrival Time.
 - **Advantage:** Minimizes average waiting time and improves overall system efficiency.
 - **Disadvantage:** Long processes may suffer from starvation if short processes continually arrive.

3. Comparison & Analysis

Comparison Focus	Priority Scheduling	SRTF

Short process with low priority	Both algorithms may suffer from starvation under certain workloads.	Executed quickly regardless of priority.
Long process vs short process	Long process dominates if its priority is high.	Long process may starve.
Fairness vs Efficiency	Based on importance (fairness).	Based on execution time (efficiency).
Starvation risk	Both algorithms may suffer from starvation under certain workloads.	Both algorithms may suffer from starvation under certain workloads.

Analysis Questions:

1. **Which algorithm produced the lower average waiting time?** SRTF generally produces a lower average waiting time because it always selects the process with the shortest remaining execution time, which reduces the waiting time for short processes.
2. **Which algorithm produced the lower average response time?** SRTF usually provides a lower average response time since shorter processes are executed earlier due to continuous preemption based on remaining time.
3. **Did priority values improve treatment of urgent processes?** Yes, Priority Scheduling improves the handling of urgent processes by always selecting the process with the highest priority first. However, this may lead to delays for low-priority processes.
4. **Did SRTF favor short jobs more aggressively?** Yes, SRTF strongly favors short jobs because any process with a shorter remaining time can immediately preempt the currently running process. However, this means both algorithms may suffer from starvation under certain workloads.
5. **Which algorithm would you recommend for the tested workload, and why?** SRTF is generally better in terms of performance metrics such as average waiting time and turnaround time. However, Priority Scheduling is more suitable when process importance is a critical factor. The final choice depends on whether the system prioritizes efficiency or process importance.

4. Required Test Scenarios & Manual Calculations

(Note: In the simulation, Priority rule is Lower Number = Higher Priority. Both algorithms are preemptive).

Scenario A: Basic Mixed Workload

A normal workload with different arrival times and burst times was used to test both algorithms under general conditions.

Process	Arrival Time	Burst Time	Priority
P1	0	5	3
P2	1	3	1
P3	2	8	4
P4	3	2	2

Calculations (For BOTH Priority and SRTF): In this specific mixed dataset, both algorithms result in the exact same execution order. P1 starts, P2 preempts P1, P4 preempts P1 upon arrival, P1 resumes, then P3 runs last.

- **Execution Order:** P1 (0-1), P2 (1-4), P4 (4-6), P1 (6-10), P3 (10-18)
- **P1:** TAT = $10 - 0 = 10$. WT = $10 - 5 = 5$. RT = 0.
- **P2:** TAT = $4 - 1 = 3$. WT = $3 - 3 = 0$. RT = $1 - 1 = 0$.
- **P3:** TAT = $18 - 2 = 16$. WT = $16 - 8 = 8$. RT = $10 - 2 = 8$.
- **P4:** TAT = $6 - 3 = 3$. WT = $3 - 2 = 1$. RT = $4 - 3 = 1$.
- **Averages:** Avg TAT = 8.0, Avg WT = 3.5, Avg RT = 2.25

CPU Scheduler Comparison Tool - Project Edition

CPU Scheduling: SRTF vs Priority (Project Workloads)

Test Scenarios:

Scenario A (Mixed)

Scenario B (Conflict)

Scenario C (Starvation)

Scenario D (Validation)

PID

Arrival

Burst

Priority

Add Process

Run & Compare

Clear All

PID	Arrival	Burst	Priority
1	0	5	3
2	1	3	1
3	2	8	4
4	3	2	2

SRTF Results (Preemptive)

P1 0-1

P2 1-4

P4 4-6

P1 6-10

P3 10-18

PID	WT	TAT	RT
1	5	10	0
2	0	3	0
3	8	16	8
4	1	3	1

Priority Results (Preemptive)

P1 0-1

P2 1-4

P4 4-6

P1 6-10

P3 10-18

PID	WT	TAT	RT
1	5	10	0
2	0	3	0
3	8	16	8
4	1	3	1

Comparison Table

Algorithm	Avg WT	Avg TAT	Avg RT
SRTF	3.5	8.0	2.25
Priority	3.5	8.0	2.25

Best (Efficiency): Priority

Scenario B: Conflict Between Burst Time and Priority

Includes a high-priority long process and a low-priority short process to reveal the difference in algorithm behavior.

Process	Arrival Time	Burst Time	Priority
P1	0	10	1
P2	1	2	5
P3	2	2	5

Priority Calculations: Execution Order: P1 has priority 1, so it runs uninterrupted. P2 and P3 tie on priority (5), so FCFS resolves the tie (P2 arrived first).

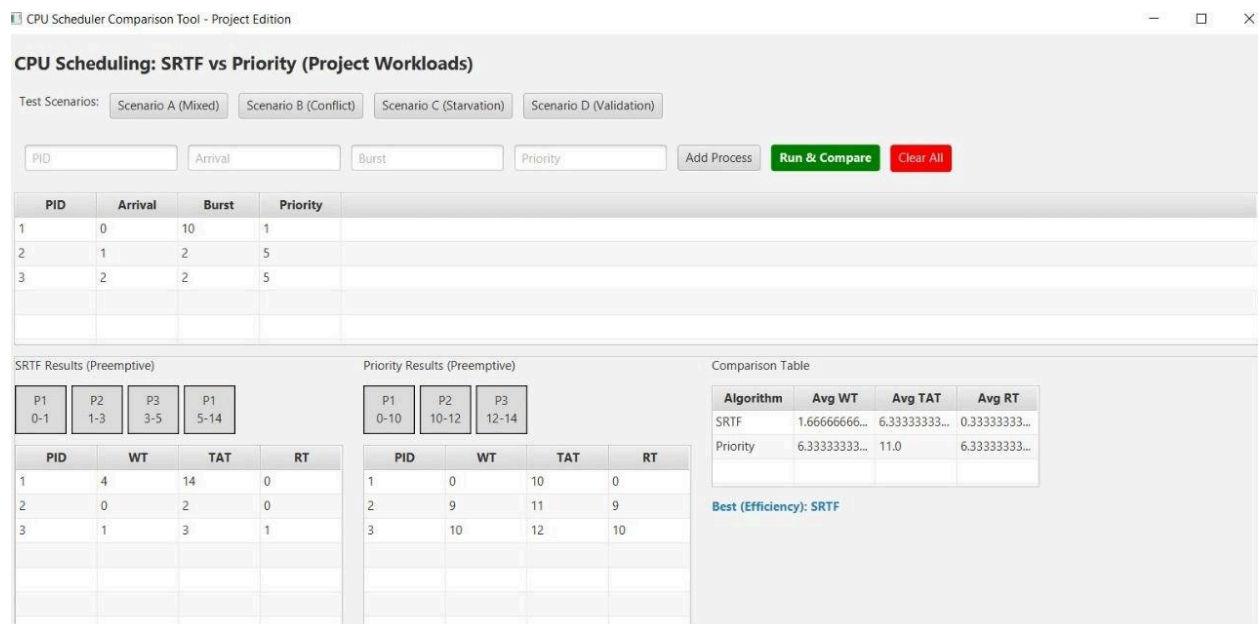
- **P1:** TAT = 10 - 0 = 10. WT = 10 - 10 = 0. RT = 0.
- **P2:** TAT = 12 - 1 = 11. WT = 11 - 2 = 9. RT = 10 - 1 = 9.

- **P3:** TAT = 14 - 2 = **12**. WT = 12 - 2 = **10**. RT = 12 - 2 = **10**.
- **Averages:** Avg TAT = **11.0**, Avg WT = **6.33**, Avg RT = **6.33**

SRTF Calculations:

Execution Order: P1 starts, P2 preempts it at time 1. P2 finishes, P3 runs. Finally, P1 resumes.

- **P1:** TAT = 14 - 0 = **14**. WT = 14 - 10 = **4**. RT = **0**.
- **P2:** TAT = 3 - 1 = **2**. WT = 2 - 2 = **0**. RT = 1 - 1 = **0**.
- **P3:** TAT = 5 - 2 = **3**. WT = 3 - 2 = **1**. RT = 3 - 2 = **1**.
- **Averages:** Avg TAT = **6.33**, Avg WT = **1.67**, Avg RT = **0.33**



Scenario C: Starvation-Sensitive Case

A scenario designed to demonstrate how both algorithms may suffer from starvation under certain workloads. P1 is a massive task with top priority, while P2, P3, and P4 are tiny tasks with terrible priority.

Process	Arrival Time	Burst Time	Priority

P1	0	20	1
P2	1	1	5
P3	2	1	5
P4	3	1	5

Priority Calculations (Starvation of Short Tasks): *Execution Order: Because P1 has the absolute maximum priority (1), it hogs the CPU for its entire 20-unit burst. P2, P3, and P4 trigger the **Aging** mechanism multiple times while waiting, raising their priorities, but they can never beat P1's top-tier priority. They must wait until P1 finishes.*

- **P1:** $TAT = 20 - 0 = 20$. $WT = 20 - 20 = 0$. $RT = 0$.
- **P2:** $TAT = 21 - 1 = 20$. $WT = 20 - 1 = 19$. $RT = 20 - 1 = 19$.
- **P3:** $TAT = 22 - 2 = 20$. $WT = 20 - 1 = 19$. $RT = 21 - 2 = 19$.
- **P4:** $TAT = 23 - 3 = 20$. $WT = 20 - 1 = 19$. $RT = 22 - 3 = 19$.
- **Averages:** Avg TAT = **20.0**, Avg WT = **14.25**, Avg RT = **14.25**

SRTF Calculations (Starvation of Long Task):

Execution Order: P1 starts. However, the moment the tiny tasks arrive, they immediately preempt P1. P2, P3, and P4 all finish rapidly, leaving P1 starved until the very end.

- **P1:** $TAT = 23 - 0 = 23$. $WT = 23 - 20 = 3$. $RT = 0$.
- **P2:** $TAT = 2 - 1 = 1$. $WT = 1 - 1 = 0$. $RT = 1 - 1 = 0$.
- **P3:** $TAT = 3 - 2 = 1$. $WT = 1 - 1 = 0$. $RT = 2 - 2 = 0$.
- **P4:** $TAT = 4 - 3 = 1$. $WT = 1 - 1 = 0$. $RT = 3 - 3 = 0$.
- **Averages:** Avg TAT = **6.5**, Avg WT = **0.75**, Avg RT = **0.0**

CPU Scheduler Comparison Tool - Project Edition

CPU Scheduling: SRTF vs Priority

Presets:
Scenario A (Mixed)
Scenario B (Conflict)
Scenario C (Starvation)
Scenario D (Validation)

PID:

Arrival:

Burst:

Priority:

Add Process

Run & Compare

Clear All

PID	Arrival	Burst	Priority
1	0	20	1
2	1	1	5
3	2	1	5
4	3	1	5

SRTF Results (Preemptive)

P1	P2	P3	P4	P1
0-1	1-2	2-3	3-4	4-23
PID	WT	TAT	RT	
1	3	23	0	
2	0	1	0	
3	0	1	0	
4	0	1	0	

Priority Results (Preemptive)

P1	P2	P3	P4
0-20	20-21	21-22	22-23
PID	WT	TAT	RT
1	0	20	0
2	19	20	19
3	19	20	19
4	19	20	19

Comparison Table

Algorithm	Avg WT	Avg TAT	Avg RT
SRTF	0.75	6.5	0.0
Priority	14.25	20.0	14.25

Best (Efficiency): SRTF

Scenario D: Validation Case

Invalid input values were tested. The system handled them safely before starting the simulation by throwing an exception alert.

- **Input Attempted:** PID = -1, Arrival = 0, Burst = 5, Priority = 1.
- **System Response:** The application safely rejected the negative PID, alerting the user to “PID must be greater than 0”.

CPU Scheduling: SRTF vs Priority (Project Workloads)

Test Scenarios:
Scenario A (Mixed)
Scenario B (Conflict)
Scenario C (Starvation)
Scenario D (Validation)

-1

0

5

1

Add Process

Run & Compare

Clear All

PID	Arrival	Burst	Priority
1	0	1	1
2	0	10	10
3	1	1	1
4	2	1	1

SRTF Results (Preemptive)

P1	P3	P4	P2
0-1	1-2	2-3	3-13
PID	WT	TAT	RT
1	0	1	0
2	3	13	3
3	0	1	0
4	0	1	0

Priority Results (Preemptive)

P1	P3	P4	P2
0-1	1-2	2-3	3-13
PID	WT	TAT	RT
1	0	1	0
2	3	13	3
3	0	1	0
4	0	1	0

Comparison Table

Algorithm	Avg WT	Avg TAT	Avg RT
SRTF	0.75	4.0	0.75
Priority	0.75	4.0	0.75

Best (Efficiency): Priority

Validation Error

PID must be greater than 0

OK

5. Final Conclusion

This project implemented and compared Priority Scheduling and Shortest Remaining Time First (SRTF) using the same set of processes under identical conditions.

1. Based on the simulation results, SRTF generally achieved better performance in terms of average waiting time, turnaround time, and response time due to its focus on executing processes with the shortest remaining time.
2. Priority Scheduling, on the other hand, ensured that higher priority processes were executed earlier, making it suitable for systems where process importance is critical.
3. The trade-off is clearly seen between efficiency and priority-based control. SRTF improves performance by reducing overall waiting time, while Priority Scheduling focuses strictly on process importance, regardless of how long the tasks take.
4. **Fairness & Starvation:** SRTF is fairer in terms of time distribution across fast tasks, while Priority Scheduling is fairer in importance-based execution. An aging mechanism was introduced to Priority Scheduling to prevent indefinite waiting. However, as observed in the tests, both algorithms may suffer from starvation under certain workloads. The choice between them depends entirely on the specific system requirements and operational objectives.